SERVICIO NACIONAL DE APRENDIZAJE - SENA
CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL
TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO)
FICHA: 3336210

DISEÑO Y DESARROLLO DE SERVICIOS WEB – PROYECTO

Especificación, Arquitectura y Codificación de las APIs RESTful para el Proyecto Formativo "Barbería Harry Estilos"
Evidencia: GA7-220501096-AA5-EV03

APRENDIZ:
Diego Fernando Caicedo Saldarriaga

INSTRUCTORA:
Verónica M. Muñoz

Colombia, 2026

1. Introducción y Justificación

En las fases previas del proyecto formativo Barbería Harry Estilos se desarrollaron los componentes front-end responsivos (GA6-220501096-AA4-EV03) y la aplicación web monolítica basada en el patrón Modelo-Vista-Controlador (MVC) utilizando Servlets, JavaServer Pages (JSP) y JDBC (GA7-220501096-AA2-EV01/EV02). Asimismo, en la evidencia GA7-220501096-AA5-EV01 se implementó un caso específico enfocado en autenticación de usuarios.

El presente documento atiende los requerimientos de la evidencia GA7-220501096-AA5-EV03: "Diseño y desarrollo de servicios web – proyecto", cuyo propósito es extender y transformar las funcionalidades centrales del proyecto formativo en un ecosistema completo de interfaces de programación de aplicaciones (APIs) bajo el estilo arquitectónico RESTful. Mediante este desacoplamiento, los módulos de Citas (Agendamiento), Catálogo de Servicios, Barberos, Clientes y Autenticación pueden ser consumidos no solo por navegadores web, sino también por futuras aplicaciones móviles Android/iOS, quioscos de autoservicio o plataformas de pago externas, garantizando alta escalabilidad, modularidad y apego a los estándares internacionales de la industria.

2. Objetivos

2.1 Objetivo General

Diseñar, codificar y documentar los servicios web RESTful requeridos para satisfacer las características operativas del proyecto formativo Barbería Harry Estilos, integrando persistencia relacional con MySQL, control de excepciones robusto, documentación viva mediante OpenAPI 3.0 / Swagger UI y versionamiento en Git.

2.2 Objetivos Específicos

- Codificar los servicios CRUD para el módulo de agendamiento de citas, permitiendo la creación, consulta detallada, actualización/reprogramación y cancelación.
- Implementar los servicios de catálogo para la consulta y administración de servicios de barbería (Corte Clásico, Barba, Paquete Completo) y del equipo de barberos profesionales.
- Construir los servicios de gestión de clientes y seguridad de acceso mediante endpoints de inicio de sesión y registro con cifrado SHA-256 y sal aleatoria.
- Generar la especificación estandarizada y la documentación interactiva en vivo utilizando Springdoc OpenAPI 3.0 y Swagger UI.
- Estructurar una colección integral de pruebas en Postman para la verificación funcional de todos los códigos de estado HTTP (200, 201, 400, 401, 404, 409).
- Administrar el ciclo de vida del código fuente utilizando control de versiones Git y publicarlo en el repositorio oficial de GitHub.

3. Arquitectura y Decisiones Tecnológicas

Para el desarrollo de las APIs se adoptó una arquitectura en capas basada en el framework Java Spring Boot 3.3.4, ejecutándose sobre Java Development Kit (JDK) 17+. Esta elección garantiza un servidor embebido de alto rendimiento (Apache Tomcat), gestión declarativa de dependencias a través de Apache Maven y serialización nativa a JSON mediante Jackson.

Capa / Componente	Tecnología Empleada	Responsabilidad en el Sistema

Controladores REST	Spring Web (@RestController)	Exposición de endpoints HTTP, validación de DTOs y control de códigos de estado HTTP.
Transferencia de Datos	Data Transfer Objects (DTO)	Encapsulamiento desacoplado de las peticiones JSON de agendamiento y credenciales.
Acceso a Datos (DAO)	JDBC con Sentencias Preparadas	Consultas seguras a MySQL con prevención de SQL Injection y mecanismo de fallback.
Base de Datos	MySQL 8.0+ / InnoDB	Persistencia relacional de citas, barberos, servicios, clientes y usuarios.
Documentación Viva	Springdoc OpenAPI 3.0 / Swagger UI	Generación automática de catálogo interactivo y esquema JSON estandarizado.
Versionamiento	Git & GitHub	Trazabilidad de cambios, control de versiones y trabajo colaborativo.

4. Matriz General de Servicios Web

A continuación se sintetiza el catálogo completo de endpoints expuestos por el servicio RESTful del proyecto formativo, especificando el método HTTP, la URI del recurso, la acción realizada y los códigos de respuesta configurados:

Método	Ruta del Endpoint	Descripción del Servicio	Códigos HTTP
GET	/api/citas	Listar todas las citas agendadas con datos completos.	200 OK
GET	/api/citas/{id}	Consultar cita específica por identificador.	200 OK, 404 Not Found

POST	/api/citas	Agendar una nueva cita (JSON en body).	201 Created, 400 Bad Request
PUT	/api/citas/{id}	Reprogramar o actualizar datos de una cita.	200 OK, 404 Not Found
DELETE	/api/citas/{id}	Cancelar y eliminar una cita del sistema.	200 OK, 404 Not Found
GET	/api/citas/barbero/{id}	Filtrar citas según el barbero asignado.	200 OK
GET	/api/servicios	Consultar catálogo completo de servicios y tarifas.	200 OK
GET	/api/servicios/{id}	Consultar información detallada de un servicio.	200 OK, 404 Not Found
POST	/api/servicios	Registrar un nuevo servicio en el catálogo.	201 Created, 400 Bad Request
PUT	/api/servicios/{id}	Modificar tarifa, duración o nombre del servicio.	200 OK, 404 Not Found
DELETE	/api/servicios/{id}	Eliminar un servicio del catálogo.	200 OK, 404 Not Found
GET	/api/barberos	Listar equipo de barberos profesionales.	200 OK
GET	/api/barberos/{id}	Consultar datos de un barbero por su ID.	200 OK, 404 Not Found
POST	/api/barberos	Registrar un nuevo barbero en el equipo.	201 Created, 400 Bad Request
GET	/api/clientes	Listar clientes registrados en el sistema.	200 OK

GET	/api/clientes/{id}	Consultar cliente por ID.	200 OK, 404 Not Found
POST	/api/clientes	Registrar nuevo cliente.	201 Created, 400 Bad Request
POST	/api/usuario/login	Autenticación de usuarios (JSON o Form).	200 OK, 401 Unauthorized, 400 Bad Request
POST	/api/usuario/registro	Registro de nuevo usuario con contraseña segura.	201 Created, 409 Conflict, 400 Bad Request

5. Especificación Técnica de Servicios

5.1 Servicio de Agendamiento de Citas (POST /api/citas)

Permite registrar una cita de barbería para un cliente determinado. El servicio recibe un objeto JSON en el cuerpo de la petición, valida la consistencia de los parámetros (barbero existente, servicio válido y fecha en formato ISO), crea o vincula el cliente automáticamente y persiste la cita asignándole el estado inicial CONFIRMADA.

Estructura de la Petición (Request Body JSON):

```
{
  "clienteNombre": "Carlos Mendoza",
  "clienteTelefono": "3109876543",
  "clienteCorreo": "carlos.mendoza@email.com",
  "barberoId": 1,
  "servicioId": 3,
  "fecha": "2026-10-28",
  "hora": "11:00",
  "estado": "CONFIRMADA"
}
```

Respuesta Exitosa (HTTP 201 Created):

```
{
  "mensaje": "Cita agendada correctamente",
  "citaId": 4,
  "cita": {
    "id": 4,
    "clienteId": 1,
    "barberoId": 1,
    "servicioId": 3,
    "fecha": "2026-10-28",
    "hora": "11:00:00",
    "estado": "CONFIRMADA",
    "clienteNombre": "Carlos Mendoza",
    "barberoNombre": "Harry",
    "servicioNombre": "Paquete Completo",
    "precio": 35000
  }
}
```

5.2 Servicio de Consulta de Citas (GET /api/citas)

Retorna la lista de todas las citas agendadas en orden cronológico inverso. La respuesta integra en un solo objeto JSON los datos relacionales mediante JOIN en base de datos (nombre de cliente, barbero asignado, nombre del servicio y precio).

5.3 Servicio de Reprogramación de Citas (PUT /api/citas/{id})

Permite actualizar la fecha, la hora, el barbero o el estado operativo de una cita existente (por ejemplo, cambiar a CANCELADA o ATENDIDA). Devuelve 200 OK con el objeto modificado o 404 Not Found si el identificador no existe en la persistencia.

5.4 Servicio de Cancelación de Citas (DELETE /api/citas/{id})

Elimina de forma permanente la cita indicada por parámetro de ruta. Retorna un mensaje confirmando la cancelación exitosa con código HTTP 200 OK, o un código 404 Not Found si no se localiza el registro.

5.5 Servicio de Catálogo de Servicios (GET /api/servicios)

Expone la oferta oficial de Barbería Harry Estilos, detallando nombre, clave única, tarifa en pesos colombianos y tiempo estimado de atención en minutos (Corte Clásico: $25.000 / 45 min; Barba: $14.000 / 25 min; Paquete Completo: $35.000 / 70 min).

5.6 Servicio de Autenticación Segura (POST /api/usuario/login)

Recibe credenciales (usuario y password) tanto en formato JSON como en parámetros de formulario x-www-form-urlencoded. Verifica la contraseña contrastando el hash SHA-256 con la sal aleatoria almacenada. Retorna HTTP 200 con el rol y nombre del usuario autenticado, o HTTP 401 Unauthorized ante credenciales erróneas.

6. Documentación Viva e Interactiva: Swagger UI

Dando cumplimiento estricto al elemento "Se debe crear la documentación de cada uno de los servicios", se integró la librería oficial Springdoc OpenAPI 3.0 en el código fuente. Esto permite a cualquier desarrollador o instructor acceder desde cualquier navegador web a:

- Interfaz Gráfica Swagger UI: http://localhost:8081/swagger-ui/index.html
- Especificación OpenAPI 3.0 JSON: http://localhost:8081/v3/api-docs
- Archivo de Especificación Exportado: DOCUMENTACION_TECNICA/OpenAPI_Specification_v3.json

A través de esta consola interactiva es posible inspeccionar los esquemas DTO de entrada y salida, visualizar los códigos de respuesta posibles y ejecutar pruebas en vivo ("Try it out") directamente contra el servidor activo.

7. Pruebas y Verificación con Postman

Se diseñó y exportó la colección oficial en formato Postman Schema v2.1.0, organizada en 5 carpetas funcionales que cubren la totalidad de los módulos desarrollados:

- 1. Módulo de Citas (Agendamiento): Listar, Buscar por ID, Agendar 201, Reprogramar PUT 200, Cancelar DELETE 200, Filtrar por Barbero.
- 2. Módulo de Servicios: Listar Catálogo 200, Buscar por ID 200, Crear Servicio 201, Actualizar Precio PUT 200.
- 3. Módulo de Barberos: Listar Equipo 200, Consultar Barbero 200, Registrar Barbero 201.
- 4. Módulo de Clientes: Listar Clientes 200, Consultar por ID 200, Registrar Cliente 201.
- 5. Módulo de Autenticación: Login Exitoso 200, Login Fallido 401, Login Incompleto 400, Registro Exitoso 201, Registro Duplicado 409.

La colección utiliza la variable de entorno {{baseUrl}} configurada en http://localhost:8081, facilitando la ejecución de pruebas automatizadas mediante el ejecutor de colecciones de Postman (Collection Runner).

8. Control de Versiones y Repositorio

El código fuente del proyecto se gestiona bajo el sistema de control de versiones Git, publicado en el repositorio remoto de GitHub:

Repositorio Oficial GitHub: https://github.com/dfcaicedosaldarriaga-cmd/barberia-harry-estilos

En dicho repositorio se registran los commits descriptivos con la evolución del proyecto desde la maquetación HTML/CSS/JS, pasando por los servlets/JSP hasta la presente implementación de las APIs REST con Spring Boot y OpenAPI.

9. Conclusiones

- Se cumplieron a cabalidad todos los requerimientos de la evidencia GA7-220501096-AA5-EV03, codificando los servicios web necesarios para cubrir el flujo de negocio del proyecto formativo Barbería Harry Estilos.
- La arquitectura RESTful adoptada separa de forma estricta las capas de presentación, lógica de negocio y persistencia, permitiendo el consumo multiplataforma seguro de los recursos.

- La incorporación de Springdoc OpenAPI 3.0 proporciona una documentación viva, interactiva y estandarizada que simplifica sustancialmente las tareas de mantenimiento, testing e integración por parte de terceros.
- El mecanismo de persistencia híbrido desarrollado (MySQL relacional con fallback en memoria) asegura la continuidad operativa y portabilidad del software en cualquier entorno de pruebas educativo o de producción.